HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

REPORT

FOOTBALL DATA ANALYSIS

Name: Nguyễn Văn Tuấn Anh

Student ID: B25DCTN009

Class: D25CQTN03-B

Mục lục

1 Giới thiệu **1**

1.1 Tóm tắt 1

1.2 Mục tiêu 1

2 Phân tích dữ liệu **2**

2.1 Thu thập và xử lý dữ liệu 2

2.2 Phân tích thống kê 2

2.3 Trực quan hóa dữ liệu 2

3 Học máy **4**

3.1 Phương pháp khuỷu tay và Silhouette 4

3.2 Giảm kích thước với PCA 4

3.3 Kết luận 6

4 Ước tính giá trị cầu thủ **7**

4.1 Thu thập dữ liệu 7

4.2 Làm sạch dữ liệu 7

4.3 Chọn cột đặc trưng 7

4.4 Tiền xử lý dữ liệu (Preprocessing Pipeline) 9

4.4.1 Nhánh thuộc tính số (Numeric Pipeline) 9

4.4.2 Nhánh thuộc tính phân loại (Categorical Pipeline) 9

4.4.3 Biến đổi biến mục tiêu 9

4.5 Xây dựng mô hình hồi quy Ridge 10

4.5.1 Huấn luyện mô hình 10

4.5.2 Đánh giá mô hình 10

4.6 Tối ưu siêu tham số bằng Grid Search 10

4.6.1 Tìm α tốt nhất 10

4.6.2 Đưa dự đoán về thang gốc 10

4.7 So sánh Ridge baseline và Ridge tối ưu 10

1. Giới thiệu

1.1 Tóm tắt

Báo cáo này khám phá dữ liệu cầu thủ từ Ngoại hạng Anh (EPL) mùa giải 2024–2025 thông qua sự kết hợp giữa phân tích dữ liệu và kỹ thuật học máy. Mục tiêu có hai mặt: (1) trích xuất và kiểm tra thông tin quan trọng về người chơi (tên, vị trí, quốc tịch, …); (2) áp dụng các thuật toán học máy để rút ra những hiểu biết và đưa ra đánh giá/dự đoán dựa trên dữ liệu, ví dụ như giá trị chuyển nhượng của cầu thủ.

1.2 Mục tiêu

Bằng cách tận dụng các phương pháp khoa học dữ liệu, phân tích này nhằm khám phá các mẫu trong thuộc tính của người chơi và thành phần đội, đồng thời xây dựng các mô hình dự đoán có thể ước lượng giá trị chuyển nhượng. Các kỹ thuật như phân cụm, hồi quy và phân loại được sử dụng để hỗ trợ việc ra quyết định dựa trên dữ liệu trong phân tích bóng đá.

2. Phân tích dữ liệu

2.1 Thu thập và xử lý dữ liệu

Bộ dữ liệu được thu thập từ FBref.com, tập trung vào những cầu thủ có hơn 90 phút thi đấu. Do website giới hạn số lượng request, nhóm sử dụng hướng tiếp cận *offline processing*: lưu cache dữ liệu theo từng đội trong mùa giải, sau đó dùng công cụ xử lý dữ liệu trên các bản cache này.

2.2 Phân tích thống kê

```
============================================================
  MP
============================================================
  TOP 3 (highest):
    Anthony Elanga                  Nottingham Forest          38.0
    Bruno Guimarães                 Newcastle United           38.0
    Bryan Mbeumo                    Brentford                  38.0
  BOTTOM 3 (lowest):
    Aaron Anselmino                 Chelsea                    0.0
    Aaron Hickey                    Brentford                  0.0
    Aaron Ramsdale                  Arsenal                    0.0

============================================================
  Playing Time_Starts
============================================================
  TOP 3 (highest):
    Bernd Leno                      Fulham                     38
    Bruno Guimarães                 Newcastle United           38
    Bryan Mbeumo                    Brentford                  38
  BOTTOM 3 (lowest):
    Aaron Anselmino                 Chelsea                    0
    Aaron Hickey                    Brentford                  0
    Aaron Ramsdale                  Arsenal                    0
```

Hình 2.1: Ví dụ thống kê với Match Play

Top 3 người chơi hàng đầu và 3 người chơi cuối cùng cho mỗi chỉ số đã được lưu trong `top3_txt`.

2.3 Trực quan hóa dữ liệu

- Chọn cột đặc trưng.

```
id_cols = ['Player', 'Team', 'Age', 'Nation', 'Position','Playing_Time_Matches','Playing_Time_Starts','Playing_Time_Minutes']
feature_cols = [col for col in df.columns if col not in id_cols]
```

Hình 2.2: Lựa chọn đặc trưng

- Xử lý các giá trị rỗng và loại bỏ ký tự phần trăm.

- Điền các giá trị thiếu và chuẩn hóa dữ liệu (Impute + Scale).

```python
def to_num(x):
    if pd.isna(x):
        return np.nan
    s = str(x).strip()
    if s == "N/a" or s == "":
        return np.nan
    s = s.replace("%","")
    return pd.to_numeric(s, errors="coerce")
```

Hình 2.3: Hàm xử lý dữ liệu

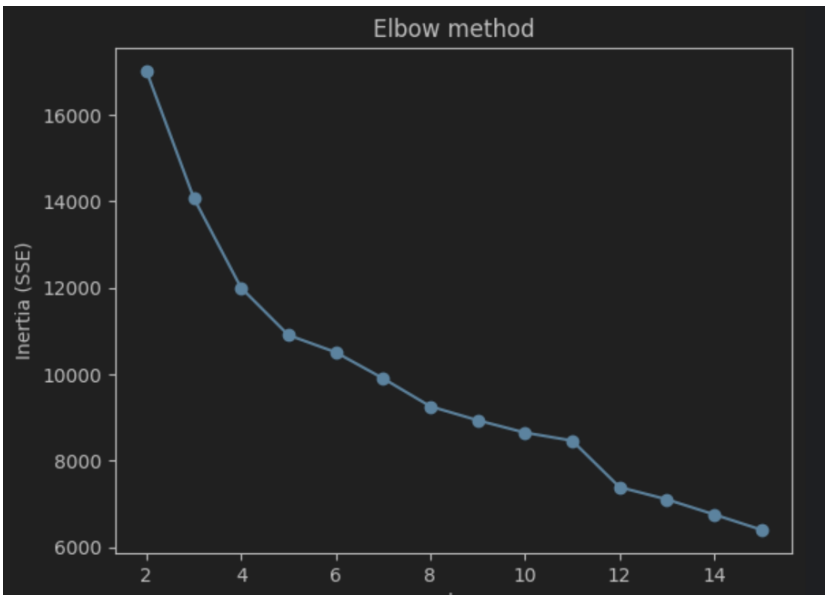
```python
imp = SimpleImputer(strategy="median")
scaler = StandardScaler()

X_imp = imp.fit_transform(X)
X_std = scaler.fit_transform(X_imp)
[45]
```

Hình 2.4: Impute (median) và StandardScaler

3. Học máy

3.1 Phương pháp khuỷu tay và Silhouette



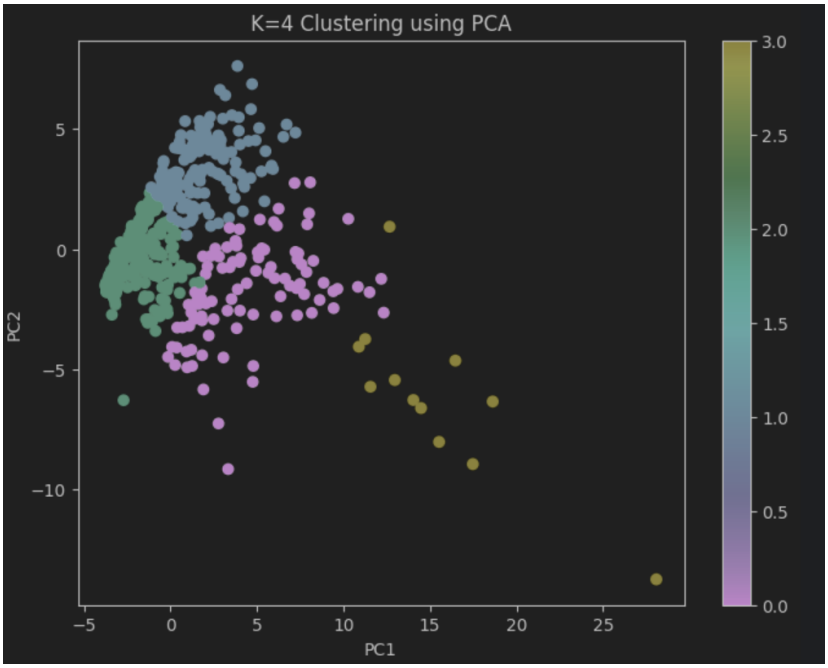
Hình 3.1: Elbow plot cho K-means

- **Chọn số cụm k theo Elbow method:** Biểu đồ Inertia (SSE) giảm nhanh khi tăng k từ 2 đến 4, cho thấy việc tăng số cụm trong giai đoạn này giúp cải thiện đáng kể độ chặt của các cụm. Tuy nhiên, từ $k \geq 4$, tốc độ giảm Inertia chậm lại và đường cong có xu hướng phẳng dần, nghĩa là lợi ích khi tăng thêm số cụm là không đáng kể so với độ phức tạp mô hình tăng lên. Do đó, điểm "khuỷu tay" xuất hiện quanh $k = 4$ và $k = 4$ được chọn là số cụm phù hợp.

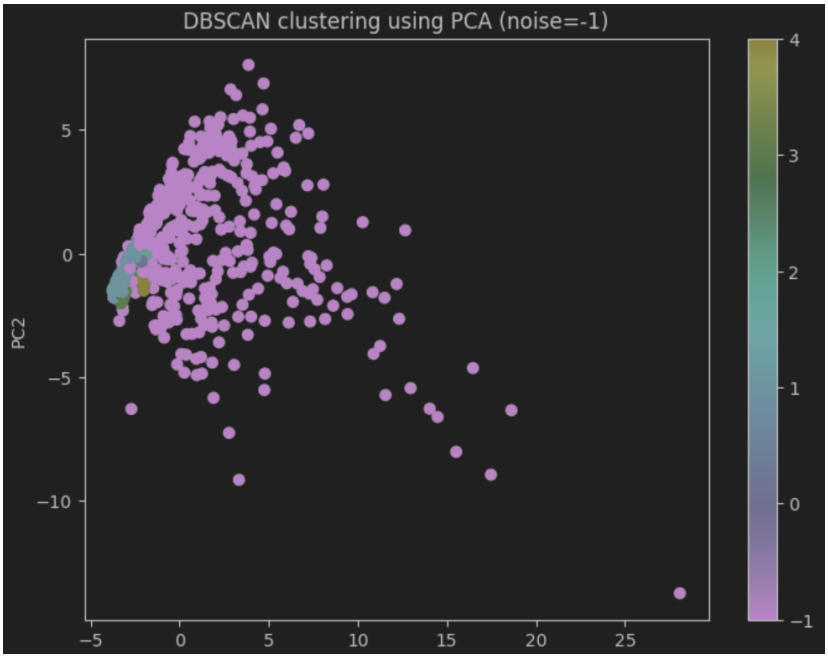
3.2 Giảm kích thước với PCA

Để trực quan hóa kết quả phân cụm, nhóm sử dụng Phân tích thành phần chính (PCA) để giảm dữ liệu thống kê của cầu thủ xuống hai thành phần chính.

- Biểu đồ PCA cho thấy các cầu thủ trong cùng một cụm có xu hướng nhóm tương đối chặt, qua đó xác thực trực quan kết quả phân cụm.
- Một số vùng chồng chéo giữa các cụm (đặc biệt giữa tiền vệ và tiền đạo) là hợp lý do tính linh hoạt vai trò trong bóng đá hiện đại.
- **So sánh với DBSCAN:** DBSCAN cho ra kết quả:
  - Số cụm: 5.
  - Số điểm nhiễu (noise): 385.



Hình 3.2: K-means ($k = 4$) sau khi giảm chiều bằng PCA



Hình 3.3: DBSCAN trên cùng bộ dữ liệu sau PCA

– Silhouette (loại bỏ noise): 0.6055.

3.3 Kết luận

- Với bộ dữ liệu này, K-means ($k = 4$) cho phân cụm rõ ràng và ổn định hơn để phục vụ mục tiêu phân nhóm tổng quát.
- DBSCAN có lợi thế phát hiện nhiễu/outlier, tuy nhiên kết quả hiện tại cho thấy tỷ lệ noise cao, chứng tỏ tham số `eps`/`min_samples` chưa phù hợp hoặc dữ liệu có mật độ không đồng đều. Vì vậy DBSCAN chưa phải lựa chọn tốt nếu mục tiêu là phân nhóm đầy đủ toàn bộ cầu thủ.

4. Ước tính giá trị cầu thủ

4.1 Thu thập dữ liệu

- Lọc danh sách cầu thủ có số phút thi đấu lớn hơn 900 phút từ file `results.csv` thành `results_min900.csv`.
- Tạo tập dữ liệu đầy đủ của mùa giải với các thuộc tính *tên* + *tuổi* + *đội bóng* + *giá trị ước tính* theo đúng định dạng và lưu vào `footballtransfers_scraped_raw.csv`.
- Ghép (*merge*) theo *tên cầu thủ* + *đội bóng* để tra cứu trong `footballtransfers_scraped_raw.csv`, bổ sung thêm một cột đặc trưng *giá trị ước tính* vào danh sách cầu thủ thi đấu trên 900 phút và lưu vào `results_min900_with_value.csv`.

4.2 Làm sạch dữ liệu

- **Chuyển đổi "Estimate value" sang dạng số:** chuẩn hoá định dạng giá trị ước tính để phục vụ bài toán hồi quy.

```python
def parse_ft_value(x):
    if not isinstance(x, str):
        return np.nan
    s = x.strip().replace(",", ".")
    s = s.replace("€", "").replace("EUR", "").strip()
    m = re.search(r"([0-9]+(?:\.[0-9]+)?)\s*(bn|b|m|k)\b", s, re.I)
    if not m:
        return np.nan
    val = float(m.group(1))
    suf = m.group(2).lower()
    if suf in ("bn","b"): val *= 1000.0
    elif suf == "k":      val /= 1000.0
    return val  # million EUR
```

Hình 4.1: Hàm chuyển đổi giá trị ước tính sang dạng số

- **Xử lý dữ liệu thiếu (missing values):** phát hiện 89 giá trị thiếu trong dataset. Nhóm loại bỏ các cột chứa missing values nhằm đảm bảo dữ liệu đầu vào đồng nhất cho mô hình.

4.3 Chọn cột đặc trưng

- Xác định tập đặc trưng đầu vào X và biến mục tiêu y (giá trị ước tính).
- Tách đặc trưng thành hai nhóm: *cột số* và *cột phân loại*.

```
print(df["Value_EUR_m"].describe())
print("Missing Value_EUR_m:", df["Value_EUR_m"].isna().sum())
[55]
count    211.000000
mean      35.945024
std       25.893434
min        1.000000
25%       14.700000
50%       38.200000
75%       49.300000
max      145.500000
Name: Value_EUR_m, dtype: float64
Missing Value_EUR_m: 89
```

Tạo Dataset để train (loại bỏ các cột có giá trị missing value)

```
df_model = df.dropna(subset=["Value_EUR_m"]).copy()
# Clean N/a values ngay từ đầu
df_model = df_model.replace(["N/a", "N/A", "NA", ""], np.nan)
df_model = df_model.infer_objects(copy=False)
print("Rows for training:", df_model.shape[0])
[56]
Rows for training: 211
```

Hình 4.2: Xử lý missing values

Chọn các cột đặc trưng và biến mục tiêu

```
target_col = "Value_EUR_m"

drop_cols = ["Player", "Nation", "Estimated Value"]
drop_cols = [c for c in drop_cols if c in df_model.columns]

feature_cols = [c for c in df_model.columns if c not in drop_cols + [target_col]]

X = df_model[feature_cols].copy()
y = df_model[target_col].copy()

print("X.shape:", X.shape)
print("y.shape:", y.shape)
[57]
X.shape: (211, 57)
y.shape: (211,)
```

Tách cột số và cột phân loại

```
cat_cols = [c for c in ["Position", "Team"] if c in X.columns]
num_cols = [c for c in X.columns if c not in cat_cols]

print("Categorical columns:", len(cat_cols))
print("Numerical columns:", len(num_cols))
[44]
Categorical columns: 2
Numerical columns: 55
```

Hình 4.3: Lựa chọn đặc trưng và phân tách cột số/cột phân loại

4.4 Tiền xử lý dữ liệu (Preprocessing Pipeline)

Nhóm xây dựng một quy trình tiền xử lý thống nhất bằng `ColumnTransformer` nhằm áp dụng các phép biến đổi khác nhau cho từng nhóm thuộc tính: (i) thuộc tính số (*numeric*) và (ii) thuộc tính phân loại (*categorical*). Cách làm này giúp dữ liệu đầu vào được chuẩn hoá đúng định dạng trước khi đưa vào mô hình học máy, đồng thời giảm nguy cơ sai sót trong quá trình xử lý.

4.4.1 Nhánh thuộc tính số (Numeric Pipeline)

Với tập thuộc tính số `num_cols`, nhóm thực hiện:

- **Bù khuyết dữ liệu:** `SimpleImputer(strategy="median")` để điền NaN bằng median của từng cột (giảm ảnh hưởng của outliers).
- **Chuẩn hoá dữ liệu:** `StandardScaler()` đưa các đặc trưng về cùng thang đo (xấp xỉ mean $= 0$, std $= 1$), giúp mô hình hội tụ ổn định hơn.

4.4.2 Nhánh thuộc tính phân loại (Categorical Pipeline)

Với tập thuộc tính phân loại `cat_cols`, nhóm thực hiện:

- **Bù khuyết dữ liệu:** `SimpleImputer(strategy="most_frequent")` điền NaN bằng mode.
- **One-hot encoding:** `OneHotEncoder(handle_unknown="ignore")` chuyển biến phân loại thành vector nhị phân và tránh lỗi khi gặp nhãn mới ở tập test.

4.4.3 Biến đổi biến mục tiêu

Nhóm áp dụng biến đổi log cho biến mục tiêu:

$$y_{\log} = \log(1 + y),$$

giúp giảm độ lệch phân phối, hạn chế ảnh hưởng của giá trị quá lớn và làm bài toán hồi quy ổn định hơn. Dạng $\log(1 + y)$ tránh lỗi khi $y = 0$.

```
preprocess = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]), num_cols),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore"))
        ]), cat_cols),
    ]
)

#log-transform target
y_log = np.log1p(y)
[45]
```

Hình 4.4: Preprocessing Pipeline với `ColumnTransformer`

4.5 Xây dựng mô hình hồi quy Ridge

Sau khi hoàn thành bước tiền xử lý, nhóm sử dụng hồi quy Ridge (*Ridge Regression*) để dự đoán giá trị cầu thủ. Ridge bổ sung điều chuẩn L_2 nhằm giảm overfitting và phù hợp khi dữ liệu có nhiều đặc trưng hoặc các đặc trưng có tương quan.

4.5.1 Huấn luyện mô hình

Nhóm kết hợp tiền xử lý và mô hình vào `Pipeline` để đảm bảo quy trình nhất quán giữa train/test và tránh rò rỉ dữ liệu (*data leakage*). Thí nghiệm ban đầu sử dụng mô hình Ridge baseline với một giá trị α cố định.

4.5.2 Đánh giá mô hình

Nhóm sử dụng các chỉ số:

- R^2: mức độ giải thích phương sai (càng cao càng tốt).
- RMSE: nhạy với sai số lớn.
- MAE: dễ diễn giải và ổn định hơn RMSE.

4.6 Tối ưu siêu tham số bằng Grid Search

4.6.1 Tìm α tốt nhất

Nhóm dùng `GridSearchCV` với kiểm định chéo 5-fold (`cv=5`) để dò tìm α theo thang log:

$$\alpha \in \{10^{-3}, \ldots, 10^{3}\},$$

tối ưu theo tiêu chí R^2. Kết quả tìm được:

- $\alpha_{\text{best}} \approx 1.778$,
- điểm R^2 tốt nhất theo cross-validation ≈ 0.637.

4.6.2 Đưa dự đoán về thang gốc

Do mục tiêu được biến đổi log, nhóm chuyển ngược dự đoán về thang gốc bằng:

$$\hat{y} = \exp(\hat{y}_{\log}) - 1,$$

(thực hiện bằng `expm1`) và tính các chỉ số trên đơn vị triệu Euro để dễ diễn giải.

4.7 So sánh Ridge baseline và Ridge tối ưu

Kết quả so sánh (tập test) giữa mô hình baseline và mô hình tối ưu theo cross-validation như sau:

Nhìn chung, mô hình tối ưu giúp **giảm MAE** (dự đoán sát hơn theo sai số tuyệt đối trung bình), tuy nhiên **RMSE tăng nhẹ** và R^2 giảm nhẹ. Điều này cho thấy tăng mức điều chuẩn

Mô hình	R^2	**MAE (triệu EUR)**	**RMSE (triệu EUR)**
Ridge baseline ($\alpha = 1$)	0.7430	10.7369	15.8555
Ridge best alpha (CV)	0.7358	10.6837	16.0772

Bảng 4.1: So sánh hiệu năng Ridge baseline và Ridge tối ưu trên tập test

có thể làm mô hình ổn định hơn ở sai số trung bình nhưng không luôn cải thiện sai số lớn.